MATLAB Assignment 2

Atrisa Hormozzadeh
--

12/2024

—

Elec1703

—

Dr Dragan Indjin

Table of Contents

1. Root finding algorithms – Bisection method

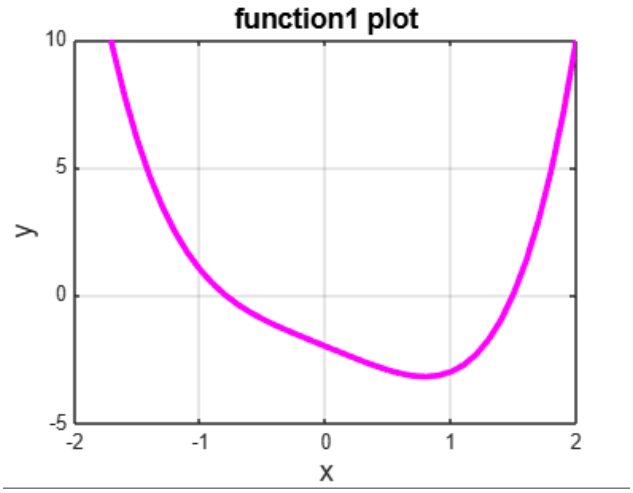
1.1 Bisection code (original)

-Function code:

/MATLAB Drive/my_function1.m

```
1  function [y]=my_function1(x)
2  y= x.^4 - 2.*x - 2;
3  end
4  %this is the code to define our function
```

-Function plot:



-Additional comment:

By looking at the graph of the function, it can be seen that near point x=-1 and x=1.5, the function changes sign and therefore has a root there. Thus, with that in mind, a proper interval width for root 1 would be from -1 to 1 and for root 2, it would be 0 to 2. (it will be discussed later what different intervals would affect our outcome).

- Testing initial bisection function with $f(x) = x^4 - 2x - 2$ with 5 iterations:

Input:

```
[zero1_bisect,error1_bisect]=mybisect(inline('my_function1(x)'),-1,1,5)
[zero2_bisect,error2_bisect]=mybisect(inline('my_function1(x)'),0,2,5)
%using the main bisection code provided from the lecture notes, we shall
%obtain the roots and the errors.The interval width and number of iteration
%can be changed manually
```

Output:

Command Window

```
zero1_bisect =

  -0.781250000000000


error1_bisect =

   0.031250000000000


zero2_bisect =

   1.468750000000000


error2_bisect =

   0.031250000000000
```

- Testing effects of interval width and number of iterations:

First, the number of iterations was changed from 5 to 10. Note that the interval width remains unchanged. the results shown in matlab are below:

```
zero1_bisect =

  -0.797851562500000


error1_bisect =

     9.765625000000000e-04


zero2_bisect =

   1.495117187500000


error2_bisect =

     9.765625000000000e-04
```

Then, the interval width was changed to a smaller scale. The original scale had a width of 2 but the new one has a width of 1. For root 1, the interval has changed from [-1 1] to [-1 0] and for root 2, it changed from [0 2] to [1 2]. The iteration number has not changed and is still n=5. The output displayed is shown:

```
zero1_bisect =

  -0.796875000000000


error1_bisect =

   0.015625000000000


zero2_bisect =

   1.484375000000000


error2_bisect =

   0.015625000000000
```

- Comments:

As seen from the results shown above, we can conclude two things regarding the number of iterations and interval width. If we increase the number of iterations, essentially, we are raising our accuracy hence why the errors calculated are smaller and our roots are more accurate. As for the interval width, when we decrease the interval width, we are making the guessing area smaller so MATLAB can run more accurate guesses in a smaller width which means better accuracy hence why the errors are smaller when the width is shortened.

1.2 Modified bisection function

- Bisection_tol code (Modified function code) :

This is the modified code written for the bisection_tol code

```
function [zeroroot niterations]=mybisect_tol(f,a,b,tol,nmax)
% this functin is midifying mybisect where it does n interations of the
% bisection method for given function.
% inputs are f (inline function), tol ( given tolerance, a,b (left
% and right edges of the interval) and nmax (maximal number of iterations)
% outputs include zeroroot (estimated value of the root) and niterations
% (number of iterations)
format long
c=f(a);
d=f(b);
if (c*d>0.0)

    error('Function has the same sign at both endpoints')
end
%disp('x y')
for i=2:nmax %1 is not valid since it will make an error
    x(i)=(a+b)/2;
    y=f(x(i));
    %disp ([x y])
    if (y==0.0) %solved the equations excatly
        niterations=1;
        break %jumps out of the loop
    end
    if (c*y)<0
        b=x(i);
    else
        a=x(i);
    end
    x(1)=0;
    x(i)=(a+b)/2;
    if (abs((x(i)-x(i-1))/x(i))<tol),break,end
end
niterations=i;
zeroroot=x(i);
%e=(b-a)/2;
end
```

-Comments:

to modify the bisection code, we need to add a new line that checks the tolerance given to the code and then keeps on doing the for loop and increasing the number of iterations until the tolerance is satisfied and reached. So by adding the line if (abs((x(i)-x(i-1))/x(i)) <tol, break,end), we are teaching MATLAB to break out of the for loop whenever the tolerance has been reached.

-Main code for bisection:

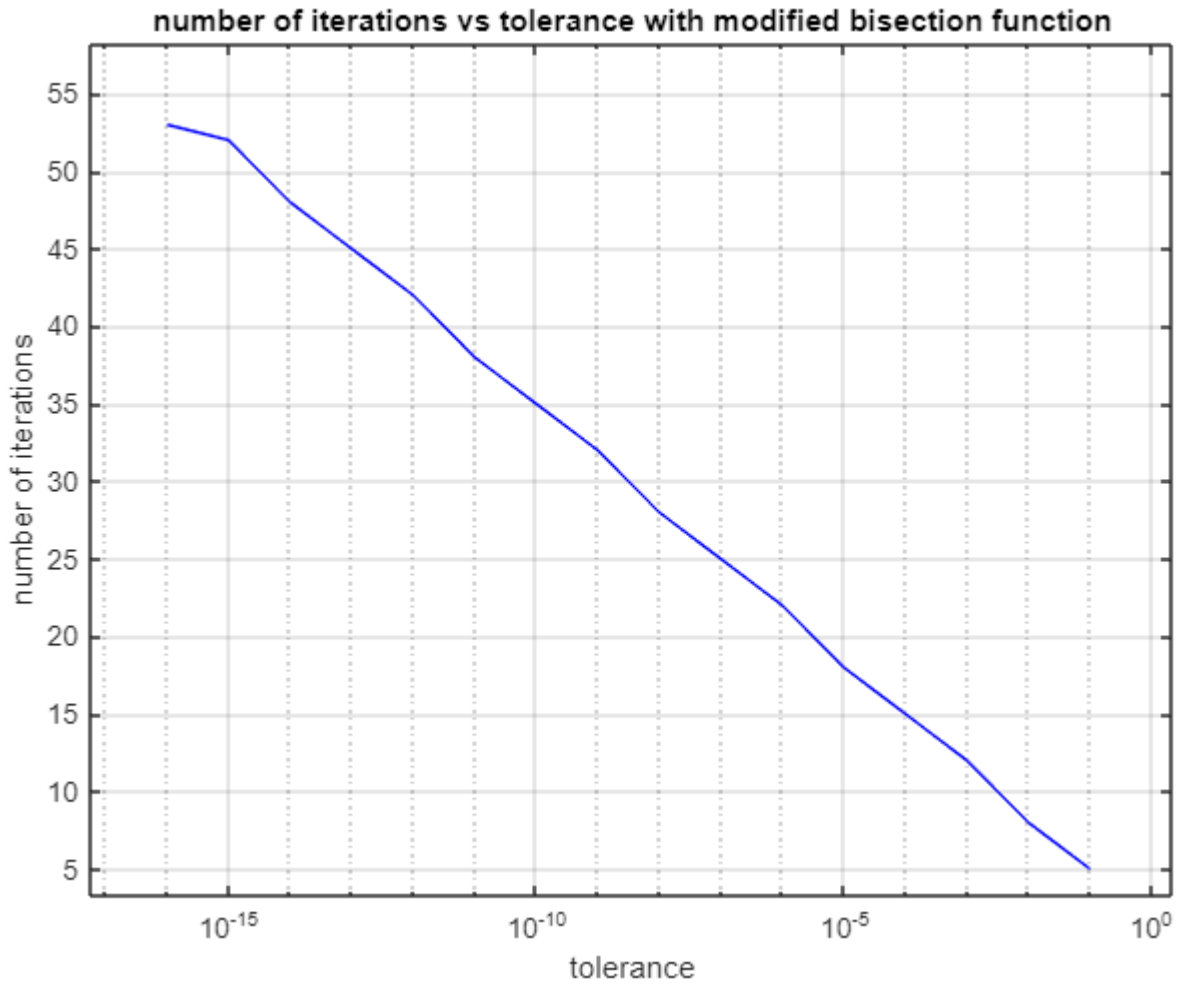
This is the main code for the next two parts.

```
clc
clear
x = -3:0.1:3;
y = my_function1(x);


[zero1 no_of_iteration1]=mybisect_tol(inline('my_function1(x)'),-1,1,1e-6,100)
[zero2 no_of_iteration2]=mybisect_tol(inline('my_function1(x)'),0,2,1e-6,100)

tolerance=10.^[-16:-1];
for k=1:length(tolerance)
    [zero1(k) no_of_iteration1(k)]=mybisect_tol(inline('my_function1(x)'),-1,1,tolerance(k),100);
end
figure(1)
semilogx(tolerance, no_of_iteration1, "b")
grid on
title('number of iterations vs tolerance with modified bisection function')
xlabel('tolerance')
ylabel('number of iterations')
```

-Test the function for tolerance 10^-6:

```
zero1 =

  -0.797623157501221


no_of_iteration1 =

    22


zero2 =

   1.494530677795410


no_of_iteration2 =

    21
```

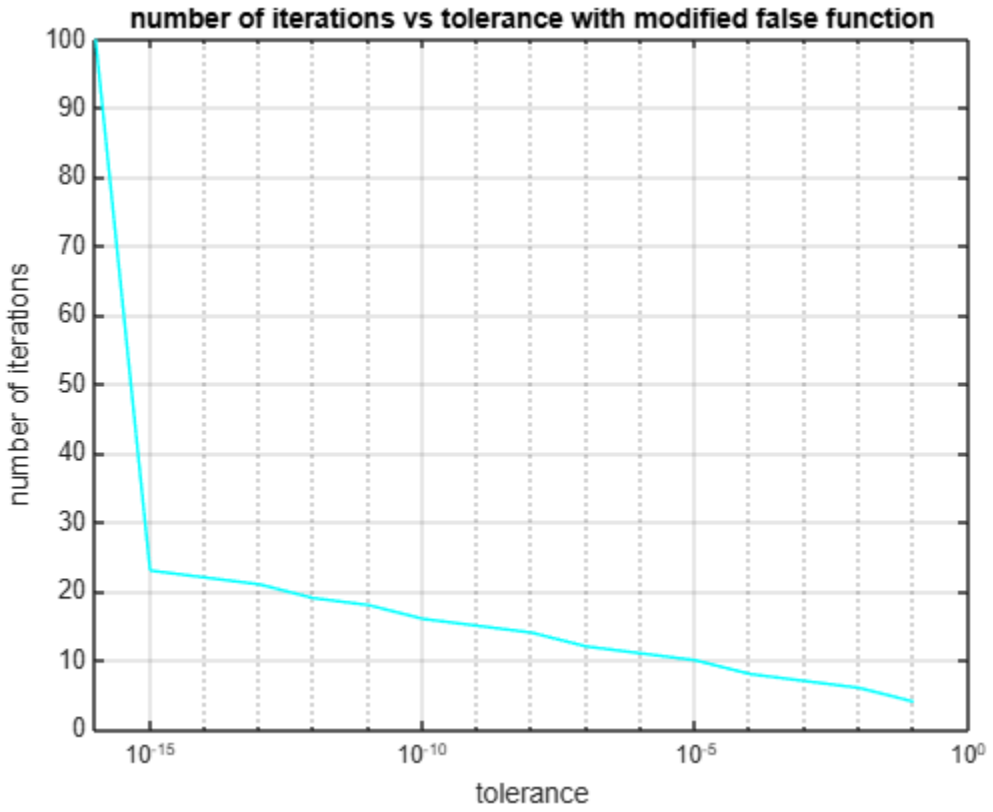
-plot of the number of iterations vs tolerance with modified bisection function:



-Main code for false:

```
clc
clear
x = -3:0.1:3;
y = my_function1(x);
plot(x,y, "m")
grid on
[zero1 no_of_iteration1]=myfalse(inline('my_function1(x)'),-1,1,1e-6,100)
[zero2 no_of_iteration2]=myfalse(inline('my_function1(x)'),0,2,1e-6,100)
%this code will provide all roots of function 1 with a lot of accuracy

tolerance=10.^[-16:-1]; % meaning = 10 ^ [-16 -15 -14 ..... -1]

for k=1:length(tolerance)
    [zero1(k) no_of_iteration1(k)]=myfalse(inline('my_function1(x)'),-1,1,tolerance(k),100)
end
figure(1)
plot(tolerance, no_of_iteration1, "r")
grid on
figure(2)
semilogx(tolerance, no_of_iteration1, "c")
grid on
```

-plot of the number of iterations vs tolerance with modified false function:



-comments:

Looking at the plots given to us, we can say that by increasing the number of iterations, we are reaching more accuracy hence the error tolerance getting more and more smaller. In bisection however the difference is in accuracy achieved with lesser number of iterations. By using the false method reaching 20 number of iterations results in a very low tolerance of 10^-15, while for bisection method the same value of tolerance is achievable for a number of iterations of almost 50. Hence why it can be concluded that the false method is more accurate. The reason for the jump in false method plot is due to the fact that after a certain tolerance value, to achieve a stricter value the number of iteration must be increased significantly.

Question 1 done.

2. Root finding algorithms – Newton method

2.1 Testing roots built in functions

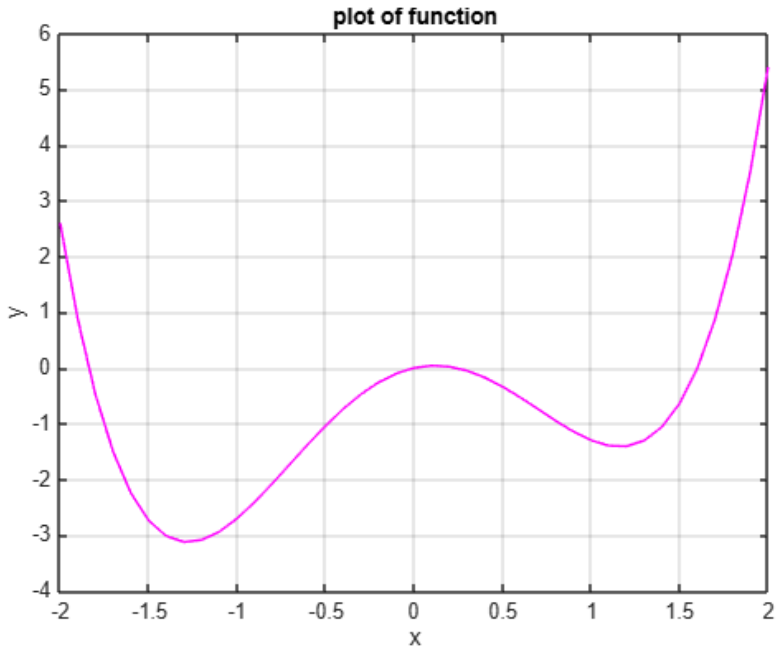
-function Code:

```
function [y]=my_function2(x)
y=x.^4-3.*x.^2+0.7.*x
end
```

-Main Code:

```
clear
clc
x=-2:0.1:2;
y=my_function2(x);
plot(x,y,"m")
grid on
title("plot pf function")
xlabel("x")
ylabel("y")
%looking at the plot given to us we can determine that this function
%have 4 real roots.
f=[1 0 -3 0.7 0]; %this line of code is a vector repesenting the polynomial
r=roots(f) %this line will calculate the roots
```

-plot of function:



-Output:

```
r =

         0
   -1.8387
    1.6009
    0.2378
```

-comments:

The roots built-in function in MATLAB gives very accurate responses for polynomial equations. However if the polynomial gets a little bit more difficult to write, it can be a confusing method to use.

2.2 Testing fzero built-in function:

-Code:

```
clear
clc
fun = @my_function2; %the function
x01=[1 2]; %the interval (note that the end points must have the same sign)
roots1=fzero(fun,x01) %the command to find the roots around interval 1
x02=[-2 -1];
roots2=fzero(fun, x02)
x03=0;
roots3=fzero(fun, x03)
x04=0.2;
roots4=fzero(fun,x04)
```

-Test 1=finding roots of $f_2(x) = x^4 - 3x^2 + 0.7x$:

```
roots1 =

    1.6009


roots2 =

   -1.8387


roots3 =

     0


roots4 =

    0.2378
```

-comments:

The objectives of this task were to learn about fzero and its limitations. Note that in the code line it has been written how the intervals must have the same sign. Fzero has a limit that when the interval provided for initial guess does not have the same sign at both ends, it will display and error which is a limit. Otherwise, fzero is very accurate.

```
Error using fzero (line 288)
Function values at the interval
endpoints must differ in sign.

Error in q2_2_test (line 7)
roots2=fzero(fun, x02)
       ^^^^^^^^^^^^^^^
```

2.3 Newtons method- finding all zeros automatically

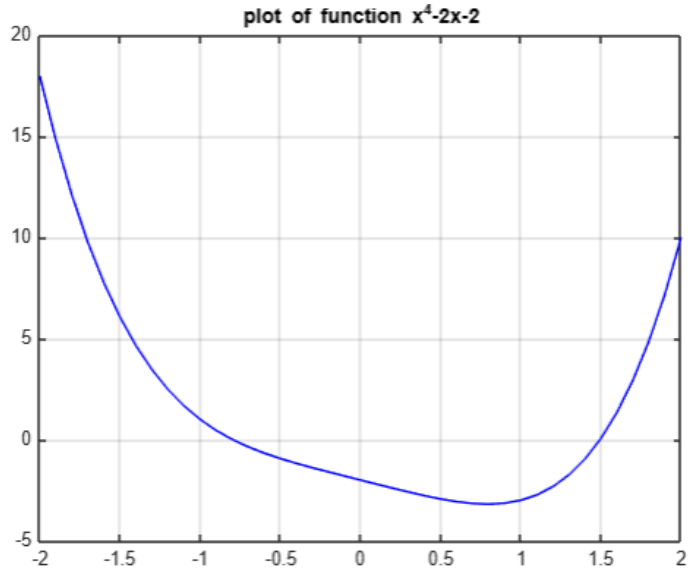
-mynewtontol function code:

```
function [x no_iterations]= mynewtontol(f,f1,x0,tol)
x = x0; % set x equal to the initial guess x0.
i=0;    % set counter to zero
y = f(x);
    while (abs(y) > tol && i < 1000)
    % Do until the tolerance is reached or max iter.
    x = x - y/f1(x); % Newton's formula
    y = f(x);
    i = i+1;
    end
no_iterations=i-1;
end
```

-Main code for filtration approach:

```
tic
clear
clc
x=-2:0.1:2;
y=my_function1(x);
plot(x,y,"b")
grid on
%to plot the function x^4-2x-2


[zero1 no_iterations1]=mynewtontol(inline("my_function1(x)"),inline("my_function1prim(x)"),-1,1e-6)
[zero2 no_iterations2]=mynewtontol(inline("my_function1(x)"),inline("my_function1prim(x)"),1,1e-6)
%these lines is the first part of the question that finds the roots
%manually

x0=linspace(-5,5,1000); %the line of array for the x-axis
for k=1:length(x0)
    [zerosfunction(k) no_iterations(k)]=mynewtontol(inline("my_function1(x)"),inline("my_function1prim(x)"),x0(k),1e-6);
end
%using this for loop, all the roots are automatically founded
zerosfunction;
zeros_rounded=round(zerosfunction,6);
zeros_final=unique(zeros_rounded) %using the unique and round operations
%(issues with these operations write here)
toc
```

-plot of the function:



-Roots array before and after filtration:

Before:

```
   1.4945    1.4945    1.4945    1.4945    1.4945    1.4945    1.4945

Columns 785 through 791

   1.4945    1.4945    1.4945    1.4945    1.4945    1.4945    1.4945

Columns 792 through 798

   1.4945    1.4945    1.4945    1.4945    1.4945    1.4945    1.4945

Columns 799 through 805

   1.4945    1.4945    1.4945    1.4945    1.4945    1.4945    1.4945

Columns 806 through 812

   1.4945    1.4945    1.4945    1.4945    1.4945    1.4945    1.4945

Columns 813 through 819

   1.4945    1.4945    1.4945    1.4945    1.4945    1.4945    1.4945

Columns 820 through 826

   1.4945    1.4945    1.4945    1.4945    1.4945    1.4945    1.4945

Columns 827 through 833

   1.4945    1.4945    1.4945    1.4945    1.4945    1.4945    1.4945

Columns 834 through 840

   1.4945    1.4945    1.4945    1.4945    1.4945    1.4945    1.4945

Columns 841 through 847

   1.4945    1.4945    1.4945    1.4945    1.4945    1.4945    1.4945
```

After:

```
zeros_final =

   -0.7976    1.4945
```

-Main code for incremental search algorithm:

```
tic
clc
clear

left_limit=-3; %defining the left interval
right_limit=3; %defining the right interval

x1=left_limit:0.1:right_limit;
y1=my_function1(x1);
plot(x1,y1,"k")
grid on

%this is the main program for the incremental test
roots_function=[];
x=linspace(left_limit,right_limit,10);
for k=1:length(x)-1
    if (my_function1(x(k))*my_function1(x(k+1))<0)
        x_mid=(x(k)+x(k+1))/2;
        [zero_new no_iterations1]=mynewtontol(inline("my_function1(x)"),inline("my_function1prim(x)"),x_mid,1e-6)
        new_root=zero_new;
        roots_function=[roots_function new_root];
    end
end
roots_function
toc %the reason why tic toc was added to this code will be discussed in the report
```

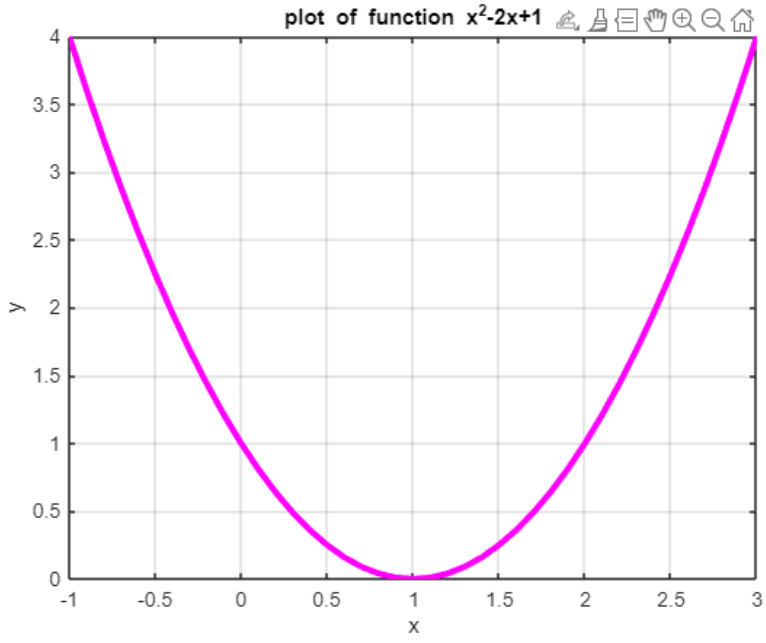
-Comments:

The main issue with the filtration approach is that after rounding to 6 decimal places, we might have turned some different answers to the same root, for examples 0.65431 and 0.65432 are different but after rounding to 3 decimal places, they are both rounded to 0.654 which would be considered the same answer. This filtration approach will make our answers less accurate.

-Roots with incremental search:

```
zero_new =

   -0.7976


no_iterations1 =

     2


zero_new =

    1.4945


no_iterations1 =

     3


roots_function =

   -0.7976    1.4945

Elapsed time is 0.032453 seconds.
>>
```

-plot of function $f(x) = x^2 - 2x + 1$*:*



-main code for $f(x) = x^2 - 2x + 1$ incremental search:

```
clc
clear
x = -1:0.1:3;
y=my_function3(x);
plot(x,y,"m", "LineWidth",3)
grid on
title('plot of function x^2-2x+1')
xlabel('x')
ylabel('y')
%this is to plot the function and find the real root, which by looking at
%the plot happens at x=1, hence this function has one root.
%now to test our code we write:
left_limit=-3;
right_limit=3;

roots_function=[];
x=linspace(left_limit,right_limit,10);
for k=1:length(x)-1
    if (my_function3(x(k))*my_function3(x(k+1))<0)
        x_mid=(x(k)+x(k+1))/2;
        [zero_new no_iterations1]=mynewtontol(inline("my_function3(x)"),inline("my_function3prim(x)"),x_mid,1e-6)
        new_root=zero_new;
        roots_function=[roots_function new_root];
    end
end
roots_function
```

-Output of the roots:

```
roots_function =

     []
```

-comments:

As seen in the outputs given by MATLAB, using the filtration approach takes much longer time than the incremental search algorithm. For the filtration approach the time calculated was 1.09 seconds (as shown below in figure1). However, for the incremental search thew time calculated was reduced to 0.074 seconds which is significantly less (see figure2)

```
zeros_final =

   -0.7976    1.4945

Elapsed time is 1.094629 seconds.
>>
```

Figure 1 = filtration approach

```
roots_function =

   -0.7976    1.4945

Elapsed time is 0.074640 seconds.
```

Figure 2= incremental search

Furthermore, for filtration the number of search was 1000 which is reasonable however if we wanted more accuracy this approach would be way more time consuming, while for incremental search even a stricter tolerance like 10^-12 takes less time than the filtration approach (see figure 3)

```
roots_function =

   -0.7976    1.4945

Elapsed time is 0.137465 seconds.
>>
```

Figure 3= incremental search with tolerance of 10^-12

-*Incremental search hazards:*

Hazzard	Example function
For functions with only one real root, the program does not display any roots. Example function: (x^2)	**See Figure 4**
For functions with no real root, the program will display no roots which can cause confusion. Example function: (x^2+4)	**See Figure 5**
For some functions that have roots in their boundary, the program might fail to recognize them. Example function: (x^2-4) at x=-2 and 2	**See figure 6**

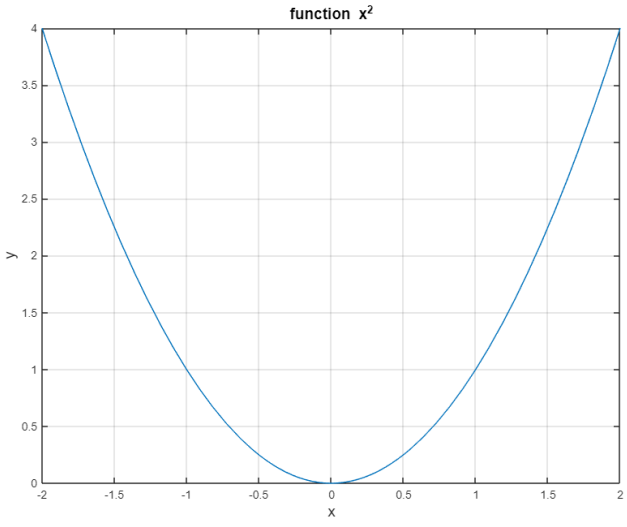


Figure 4= plot of function x^2

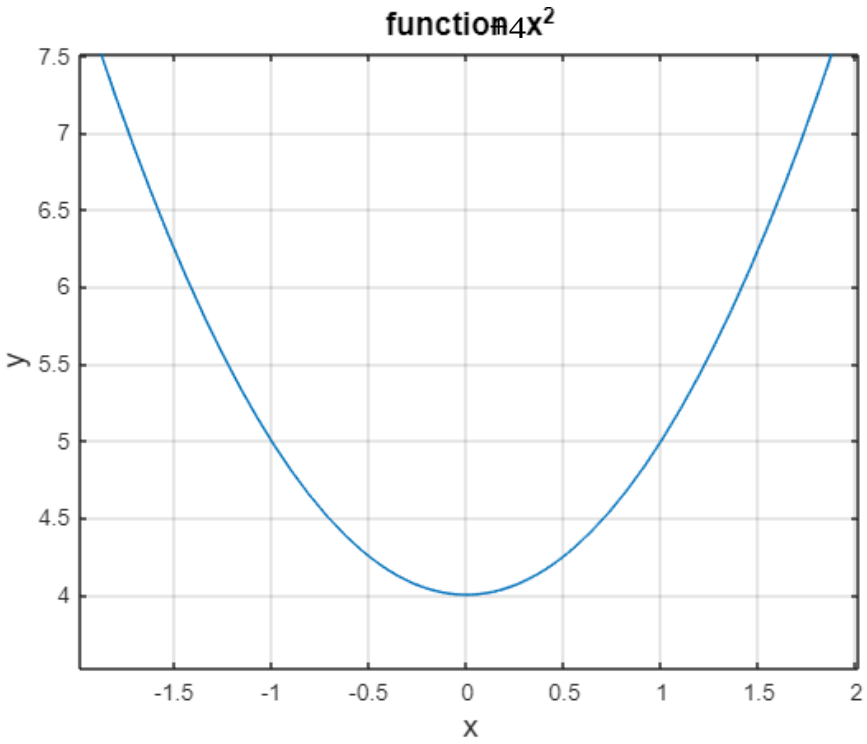


Figure 5= plot of function x^2+4

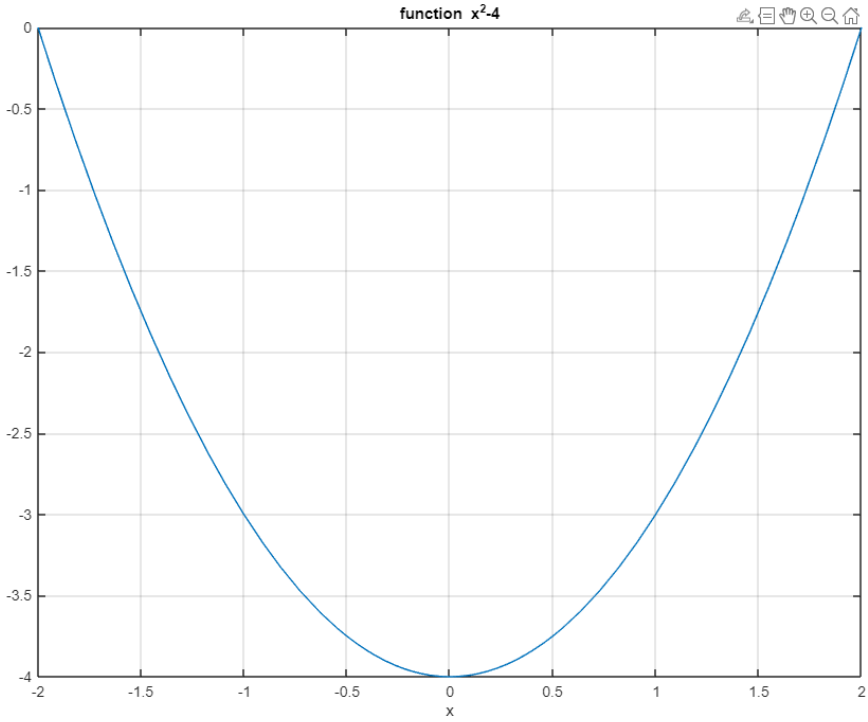


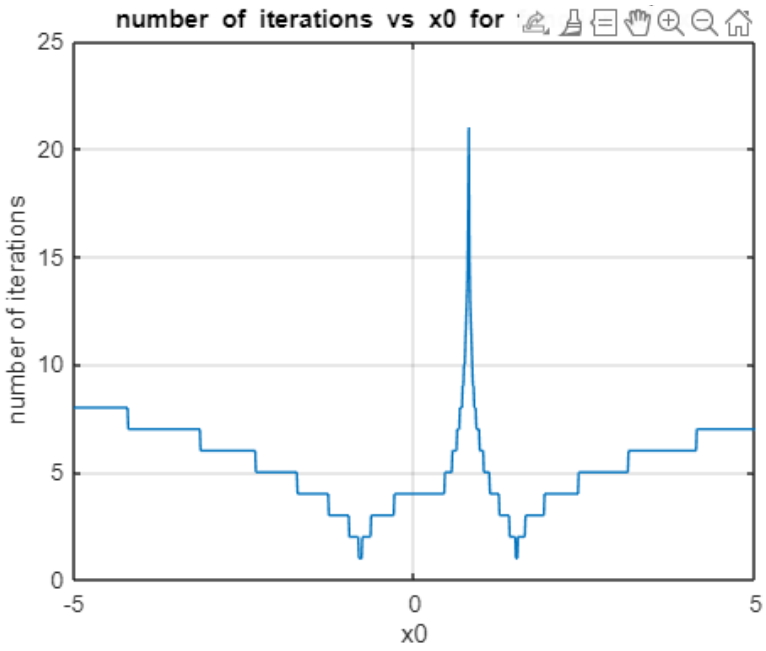
Figure 6= plot of function x^2-4

2.4 Newtons method- effect of x

-Main code:

```
clc
clear
x=-3:0.1:3;
y=my_function1(x); %calling our function x^4-2x-2
y_prim=my_function1prim(x);
y_div_yprim= y./y_prim; %this is the newton function f(x)/f'(x)
figure(1)
plot(x,y,x,y_prim)
title('plot of the functoin and its derivitive')
legend('function', 'driv of function')
xlabel('x')
ylabel('y')
grid on
%the nect 4 line of codes are our main program
x0=linspace(-5,5,1000); %the line of array for the x-axis
for k=1:length(x0)
    [zerosfunction(k) no_iterations(k)]=mynewtontol(inline("my_function1(x)"),inline("my_function1prim(x)"),x0(k),1e-6);
end

figure(2)
plot(x0,no_iterations)
grid on
title('number of iterations vs x0 for function x^4-2x-2')
xlabel('x0')
ylabel('number of iterations')

figure(3)
plot(x,y_div_yprim)
title('newton theory')
xlabel('x')
ylabel("f(x)/f'(x)")
grid on
```

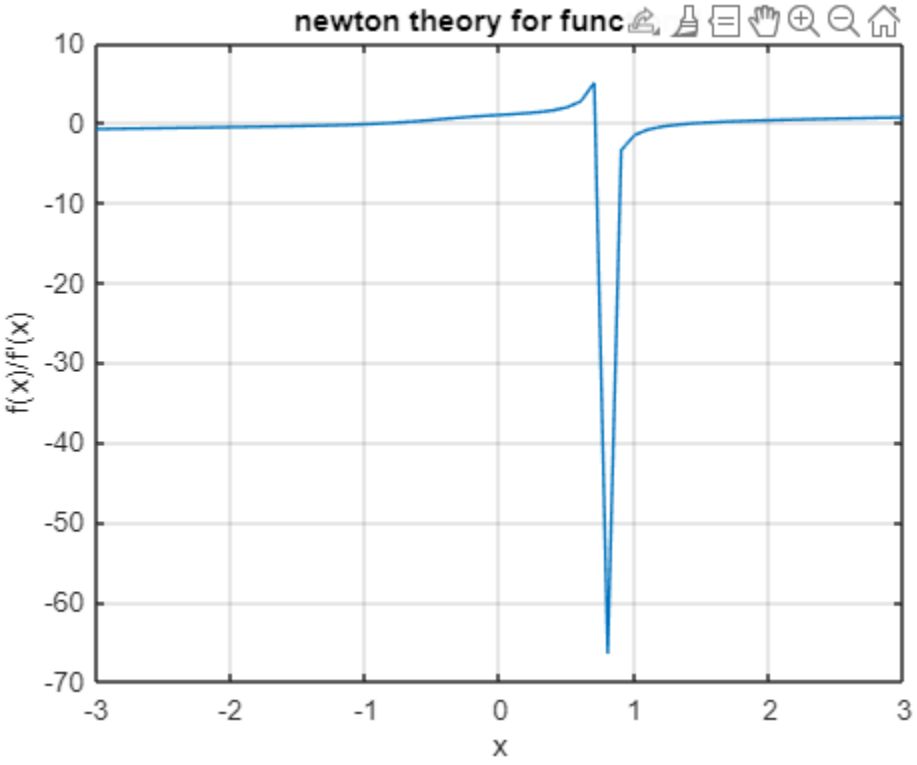
-plot of number of iterations vs x0 for function 1:



-comment:

By looking at the plot above, it can be observed that when our initial guess x0 is close to the real value of the root, the number of iterations needed jumps down to lees than 5. However, it can also be seen that when x0 reached a certain number the number of iterations needed to calculate the roots jumps to more than 20. The reason for this will be discussed after the newton theory has been observed. Other than that it can be said that the closer we get to the real value of the roots the lees number of iterations needed to get the correct answer with high accuracy.

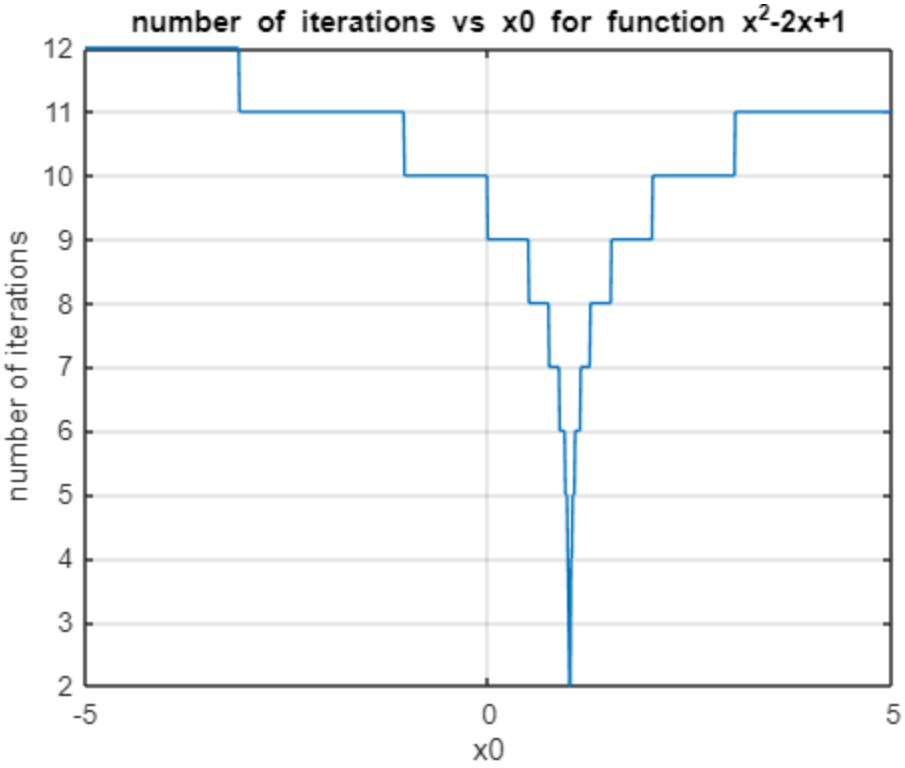
-plot of f(x)/f'(x):



-comment:

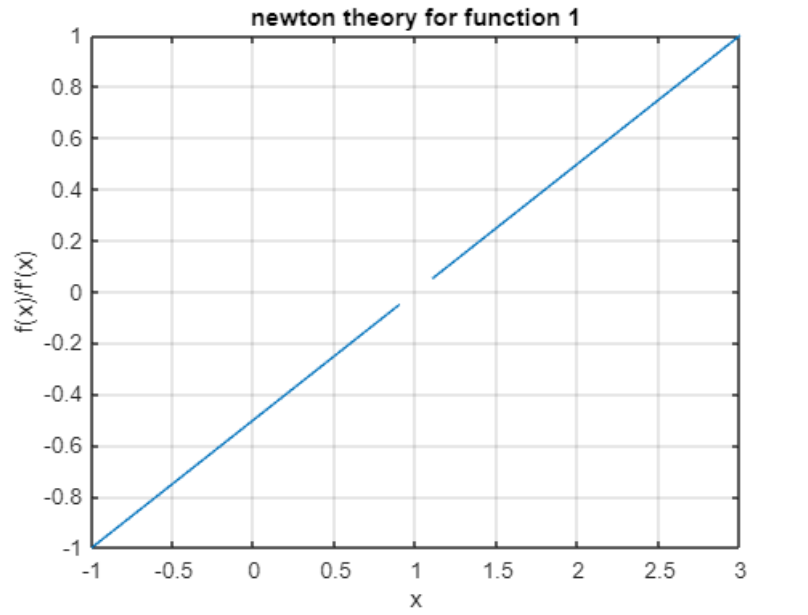
As seen in this graph, when x approaches a certain value there is a jump in the plot. This jumps happens due to f'(x) having a root at that value (approximately x=0.8) since number/0 is not defined, this jumps occurs. The reason why at a certain value as discussed above the number of iterations has a jump to more than 20 is also due to this reason.

-plot of number of iterations vs x0 for function x^2-2x+1:



-Comment:

Again, as seen in the previous graphs, we have a jump when x0 approaches the true value of the root which is one. This function at x=1, does not have a defined f(x)/f'(x), which is why around x=1 the number of iterations act like a negative infinity graph meaning the more we get closer to x0=1, the smaller our number of iterations gets but never getting to the actual value of x0=1. Below you can see a graph of this function's newton theory.

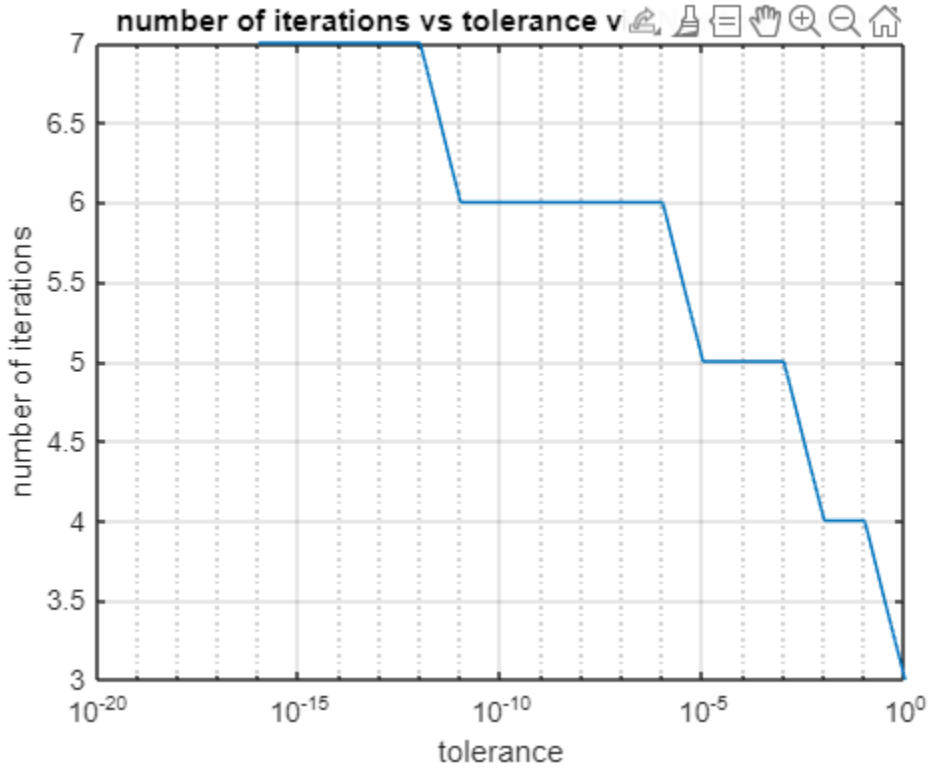


2.5 Newtons method- effect of number of iterations:

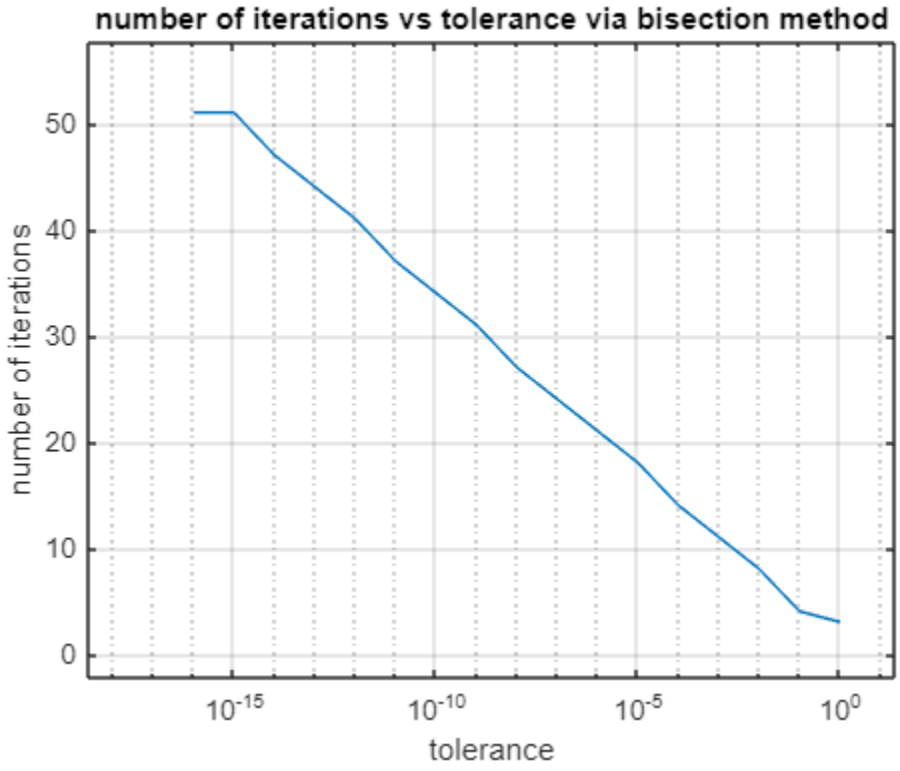
-Main code:

```
clc
clear
x=-3:0.1:3;
y=my_function1(x);%recalling the function x^4-2x-2
y_prim=my_function1prim(x);
y_div_yprim= y./y_prim;
figure(1)
plot(x,y,x,y_prim)
grid on

x0=1; %this line fixes the initial guess
tolerance=10.^[-16:0];
%main program for newton method
for k=1:length(tolerance)
    [zerosfunction(k) no_iterations(k)]=mynewtontol(inline("my_function1(x)"),inline("my_function1prim(x)"),x0,tolerance(k));
end

tolerance=10.^[-16:0];
%main program for bisectoin method
for k=1:length(tolerance)
    [zero1(k) no_of_iteration1(k)]=mybisect_tol(inline('my_function1(x)'),1,3,tolerance(k),100);
end

figure(2)
semilogx(tolerance, no_iterations)
grid on
title('number of iterations vs tolerance via Newton method')
xlabel('tolerance')
ylabel('number of iterations')

figure(3)
semilogx(tolerance, no_of_iteration1)
grid on
title('number of iterations vs tolerance via bisection method')
xlabel('tolerance')
ylabel('number of iterations')
```

-plot of number of iterations vs tolerance via newton method:



-plot of number of iterations vs tolerance via Bisection method:



-Comments:

The first thing noticeable is that to achieve a low and strict tolerance we must increase our number of iterations in both methods. For example, in bisection method to achieve a tolerance of 10^-15 (a very strict tolerance) we must have more than 50 number of iterations. By comparing the values given to us in the graphs, it can be concluded that the newton method is far more accurate than bisection. To achieve the strict tolerance of 10^-16 via newton method we only need 7 number of iterations. And even with 5 number of iterations we still have a reasonable tolerance of 10^-5. However, to achieve this tolerance via bisection method, we must increase our number of iterations to almost 20 which is time consuming. In conclusion, it can be said with confidence that the newton method is far more accurate and less time consuming than the bisection method.

Question 2 done.

3. function fitting – linear and nonlinear regression

3.1 Linear least squares regression fitting

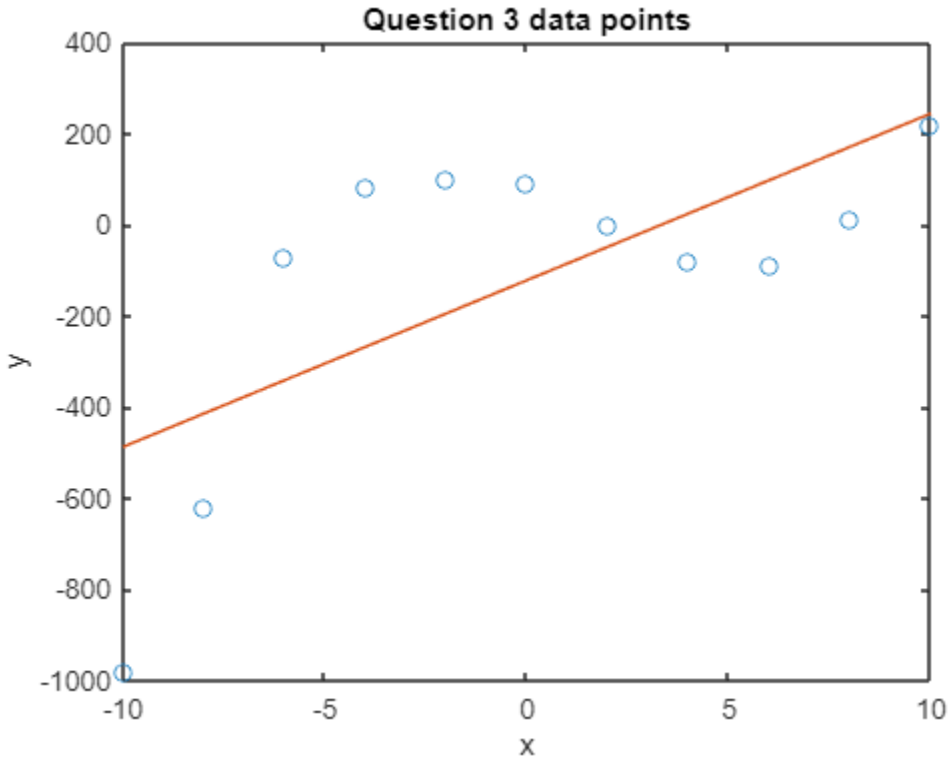
-mylinreger function code:

```
function [a, r2] = mylinregr(x,y)
% linregr: linear regression curve fitting
%   [a, r2] = linregr(x,y): Least squares fit of straight
%           line to data by solving the normal equations
% input:
%   x = independent variable
%   y = dependent variable
% output:
%   a = vector of slope, a(1), and intercept, a(2)
%   r2 = coefficient of determination
n = length(x);
if length(y)~=n, error('x and y must be same length'); end
    x = x(:); y = y(:); % convert to column vectors
    sx = sum(x); sy = sum(y);
    sx2 = sum(x.*x); sxy = sum(x.*y); sy2 = sum(y.*y);
    a(1) = (n*sxy-sx*sy)/(n*sx2-sx^2); %calculate slope
    a(2) = sy/n-a(1)*sx/n;  %calculate intercept
    r2 = ((n*sxy-sx*sy)/sqrt(n*sx2-sx^2)/sqrt(n*sy2-sy^2))^2; %r2 coefficient
% create plot of data and best fit line
%xp = linspace(min(x),max(x),2);
%yp = a(1)*xp+a(2);
%plot(x,y,'o',xp,yp)
%grid on
end
```

-Main code:

```
clc
clear

x=[-10 -8 -6 -4 -2 0 2 4 6 8 10]; %defining arrays of x
y=[-980 -620 -70 80 100 90 0 -80 -90 10 220]; %defining arrays of y


[a r2]=mylinregr(x,y) %where a is our slope and intersect and r is regression

xp = linspace(min(x),max(x),100);
yp = a(1)*xp+a(2); %a(1) here is a0 and a(2) is a1 from the assignment
plot(x,y,'o',xp,yp)
title('Question 3 data points')
xlabel('x')
ylabel('y')
```

-plot of fitted data:



-Output:

```
a =

   1.0e+02 *

   0.365454545454545  -1.218181818181818


r2 =

   0.461277616030142

>>
```

-comments:

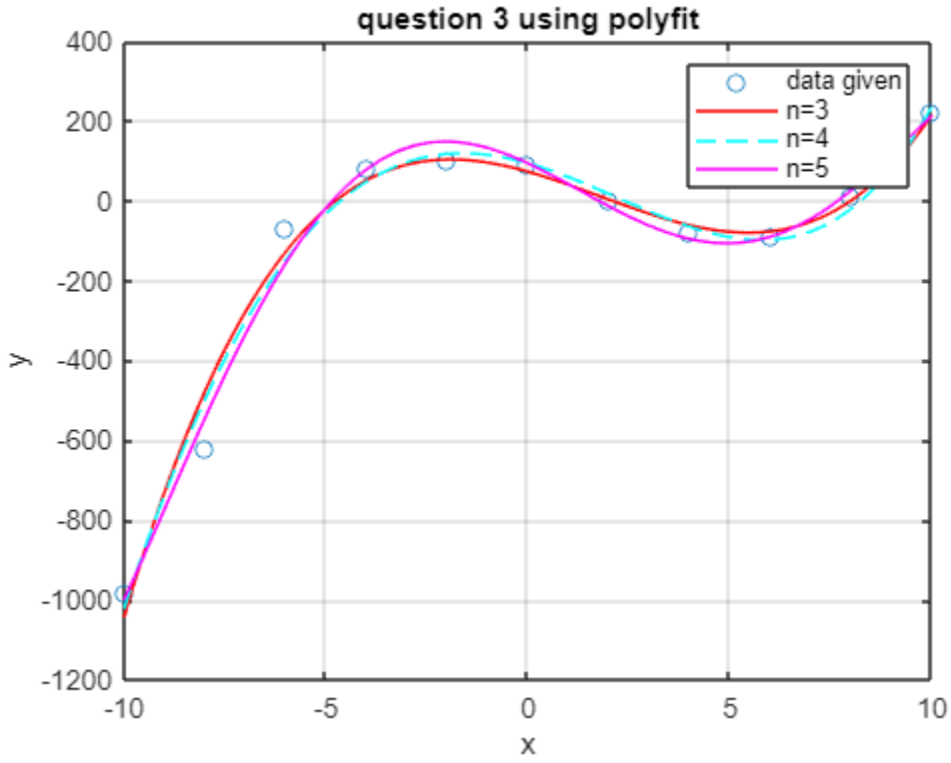
By analyzing the regression value calculated by MATLAB, which is r=0.46, it can be said that the data points given to us are scattered too much and the best line fitting is not very accurate as a good regression value would be 0.9. By observing the plot and the best line fit, we can conclude that our data are not accurate and at the time of calculating them there has been some mistakes.

3.2 Polynomial Fitting:

-Main code:

```
clc
clear

x=[-10 -8 -6 -4 -2 0 2 4 6 8 10];%defining arrays of x
y=[-980 -620 -70 80 100 90 0 -80 -90 10 220];%defining arrays of y

figure(1)
plot(x,y)
grid on

%by using polyfit and n=3,4,5 we will create three different plots, eahc
%more accurate than the last one.
p=polyfit(x,y,3);
xx=min(x):0.01:max(x);
yy=p(1)*xx.^3+p(2)*xx.^2+p(3).*xx+p(4);

p4=polyfit(x,y,4);
xx4=min(x):0.01:max(x);
yy4=p4(1)*xx4.^4+p4(2)*xx4.^3+p4(3).*xx4.^2+p4(4)*xx4+p4(5);

p5=polyfit(x,y,5);
xx5=min(x):0.01:max(x);
yy5=p5(1)*xx5.^5+p5(2)*xx5.^4+p5(3).*xx5.^3+p5(4)*xx5.^2+p5(5)*xx5+p5(6);

plot(x,y,"o", xx, yy, "-r", xx4, yy4, "--c", xx5, yy5, "-m")
```

-plot:



-Comments:

By observing the plot above, it can be seen that by increasing n (degree) the curve gets more and more accurate, hence why the curve given to us by n=5 is the best fitted and the most accurate. So, by increasing n we increase our accuracy and the curve plotted is closer to the actual data points. (As seen in the graph, where n=5 has the closest curve to the data points)

Question 3 done.

4. Interpolation

4.1 Lagrange interpolation

-Lagrange interpolation function code:

```
function yint = lagrange(x,y,xx)
% Lagrange: Lagrange interpolating polynomial
%   yint = Lagrange(x,y,xx): Uses an (n - 1)-order
%     Lagrange interpolating polynomial based on n data points
%     to determine a value of the dependent variable (yint) at
%     a given value of the independent variable, xx.
% input:
%   x = independent variable
%   y = dependent variable
%   xx = value of independent variable at which the
%        interpolation is calculated
% output:
%   yint = interpolated value of dependent variable
n = length(x);
if length(y)~=n, error('x and y must be same length'); end
s = 0;
 for i = 1:n
   product = y(i);
   for j = 1:n
     if i ~= j
      product = product.*(xx-x(j))./(x(i)-x(j));
     end
   end
   s = s+product;
 end
 yint = s;
end
```

-Main code:

```
clc
clear

x1=1920:10:2000; %defining x data
y1=[105 120 130 150 180 205 225 250 280]; %defining y data

xx=1925;
yy=lagrange(x1,y1,xx) %number of population in year 1925 estimated by lagrange

xx1=1945;
yy1=lagrange(x1,y1,xx1)%number of population in year 1945 estimated by lagrange
p=polyfit(x1,y1,1);
regression=polyval(p,x1);
x0=1945;
regression_1945=polyval(p,x0) %the result is 149 million where
% lagrange gave us 137 million
diff_1945=abs(regression_1945-yy1)

xx0=1920:5:2000;
yy0=my_lagrange(xx0);
yy0_lin=p(1)*xx0+p(2);

y_2015=my_lagrange(2015)%number of population in year 2015 estimated by
% using the program my_lagrange


figure(1)
hold on
stem(x1,y1) %as instructed, using stem to plot this function
title('Population of a region')
xlabel('year')
ylabel('population in million')
plot(x1,y1,'o',xx0,yy0,"c",xx0,yy0_lin,'m')
title('best fitting line for given data')
xlabel('year')
ylabel('population in million')
```

-Population in 1945- difference between Lagrange interpolation and linear fitting:

Code for linear:

```
p=polyfit(x1,y1,1);
regression=polyval(p,x1);
x0=1945;
regression_1945=polyval(p,x0) %the result is 149 million where
% lagrange gave us 137 million
```

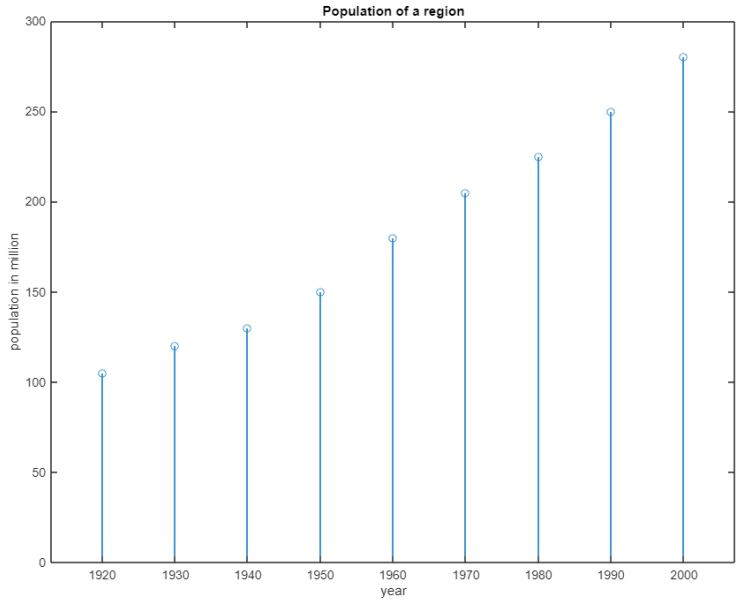
output:

```
yy1 =

     1.379765319824219e+02


regression_1945 =

     1.494027777777783e+02


diff_1945 =

  11.426245795356408
```

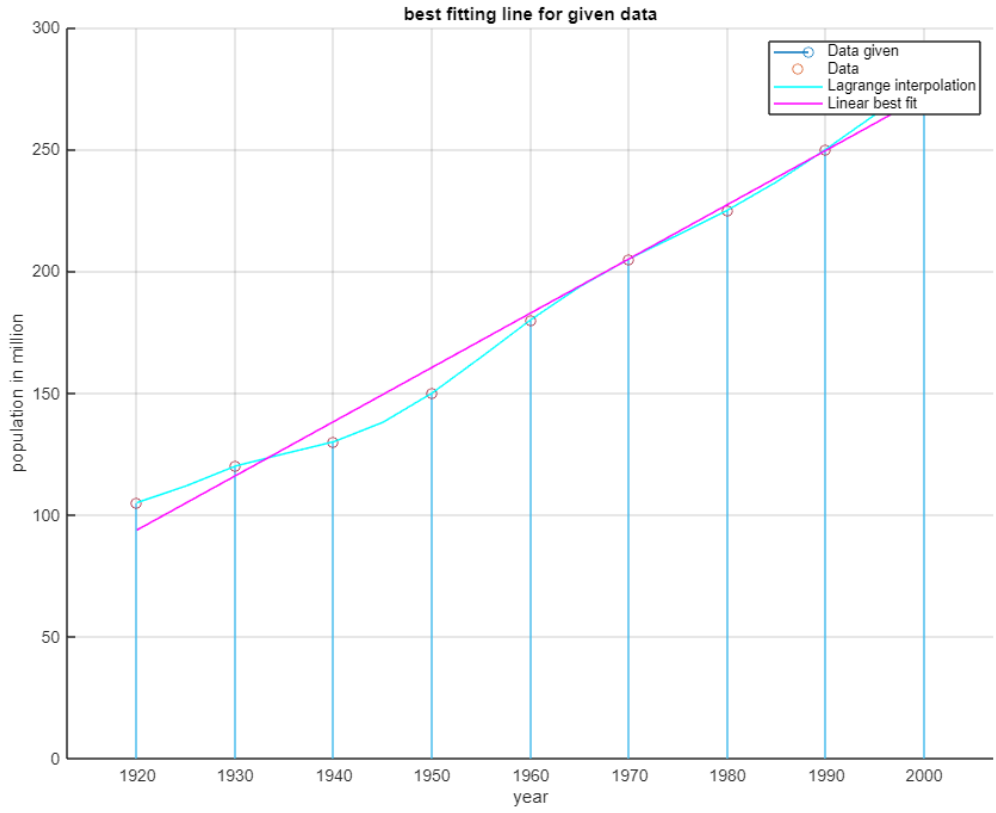
-Population in 1925:

```
yy =

    1.118730163574219e+02
```

-Plot of the data:



-Plot of fitted data be Lagrange interpolation and linear fit:



Atrisa Hormozzadeh

-Population of 2015 by Lagrange:

```
y_2015 =

     5.582475280761719e+02
```

-my_Lagrange function code:

```
function [y]=my_lagrange(x)
x1=1920:10:2000;
y1=[105 120 130 150 180 205 225 250 280];
y=lagrange(x1,y1,x);
end
```

-Comments:

While the best fitted line is very accurate, it can be said that Lagrange is more accurate and closer to the data by observing the plot above. The difference between the two method is 11 years for year=1945 which is not a big difference if high accuracy is not our main aim. However, if we are pushing for a more accurate result, Lagrange is the better option.

4.2 Inverse interpolation

a)200 million

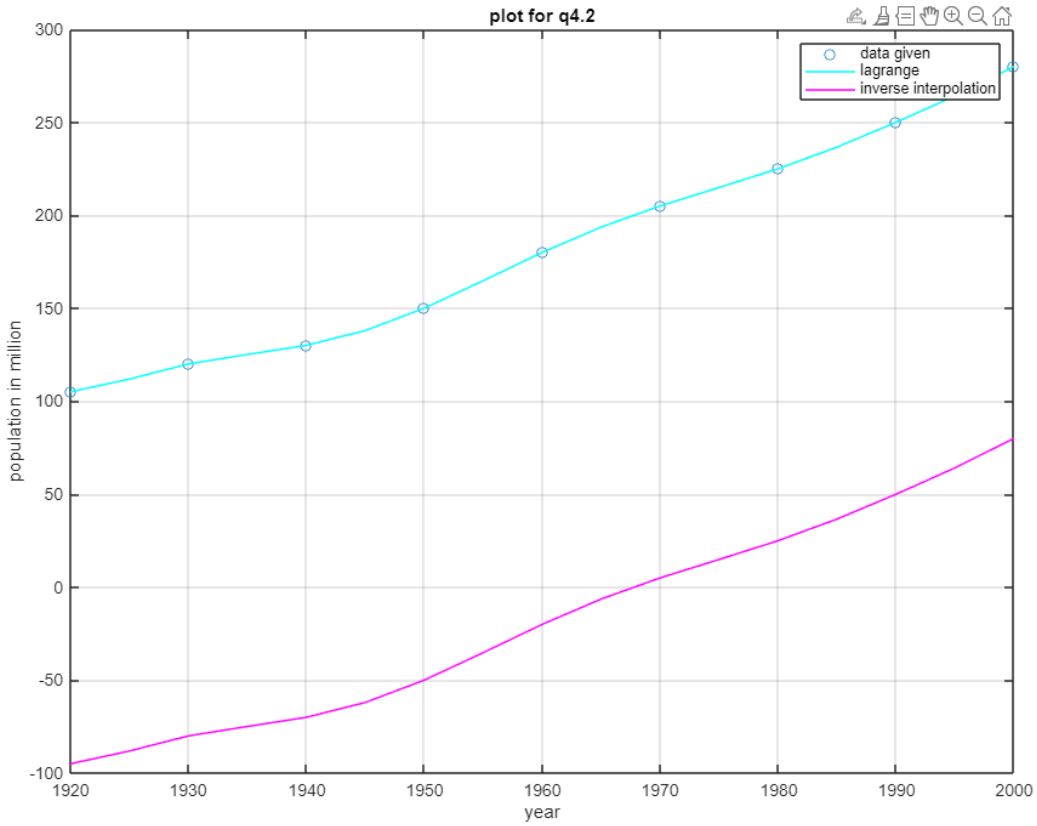
-my_lagrangeinv Code function:

```
function [y]=my_lagrangeinv(x)
x1=1920:10:2000;
y1=[105 120 130 150 180 205 225 250 280];
y=lagrange(x1,y1,x)-200;
end
```

-Main code:

```
clc
clear
x1=1920:10:2000; %defining x data
y1=[105 120 130 150 180 205 225 250 280]; %defining y data


xx=1925;
yy=lagrange(x1,y1,xx)

xx1=1945;
yy1=lagrange(x1,y1,xx1)

xx0=1920:5:2000;
yy0=my_lagrange(xx0);



yy2=my_lagrangeinv(xx0);
figure(2)
plot(x1,y1,'o',xx0,yy0,"c", xx0, yy2, "-m")
grid on
title('plot for q4.2')
legend('data given','lagrange','inverse interpolation')
xlabel('year')
ylabel('population in million')

%4.2)a
root_200=fzero('my_lagrangeinv',1970) %the root of the inverse function which is year 1967
% , month 7

%4.2)b
Quad_reg= polyfit(x1, y1, 2); % Quadratic regression
g = @(x) lagrange(x1, y1, x) - polyval(Quad_reg, x);
intersection_year = fzero(g, [1920, 2000])
```

-plot of function for inverse interpolation:



-Year of 200 million:

This is the output which gives us the year 1967(near month 7).

```
root_200 =

    1.967686439967592e+03
```

-Comments:

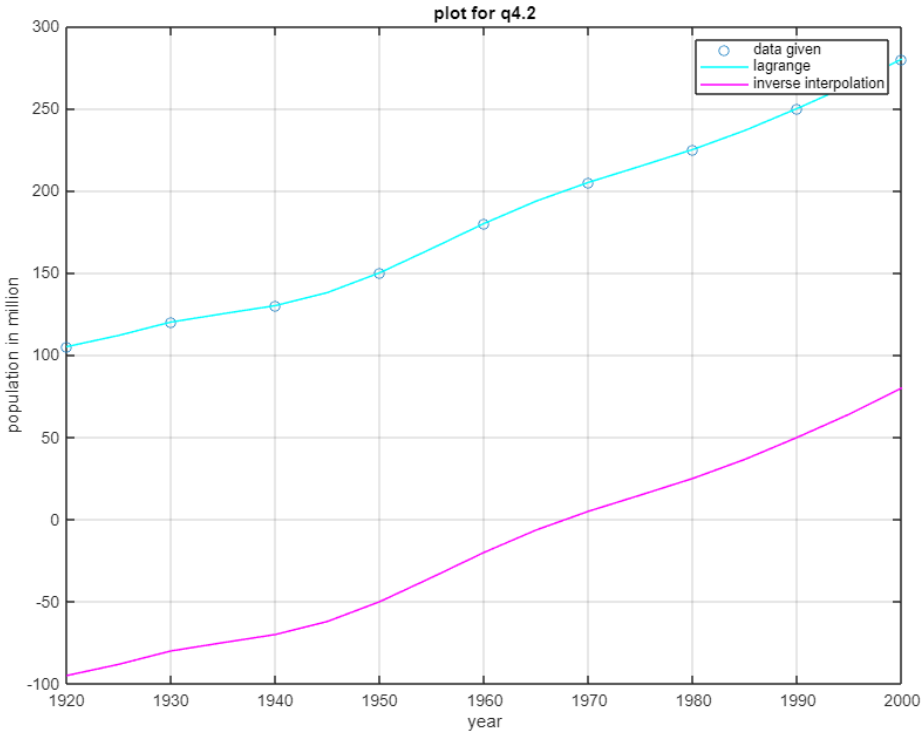
The inverse interpolation function was created by lowering the original Lagrange interpolation plot by 200 so that the root of the new function happens when the original function has the exact value of 200 million which is why we find the root of the new plot to get our answer which year 1967.

b) Lagrange and quadratic regression comparison

-code:

```
%4.2)b
Quad_reg= polyfit(x1, y1, 2); % Quadratic regression
g = @(x) lagrange(x1, y1, x) - polyval(Quad_reg, x);
intersection_year = fzero(g, [1920, 2000])
```

-plot:



-year when two fitting method yield same results:

```
intersection_year =

    1.980676366040816e+03
```

-comment:

Overall, by observing the plots and the differences between the answers it can be said that inverse interpolation is a better and more accurate method that linear fit. The issue with linear fit is also its complexity whereas interpolation is more straightforward.

Question 4 done.

5. Numerical integration

5.1 Trapezoid integration

-Trap 2 function code:

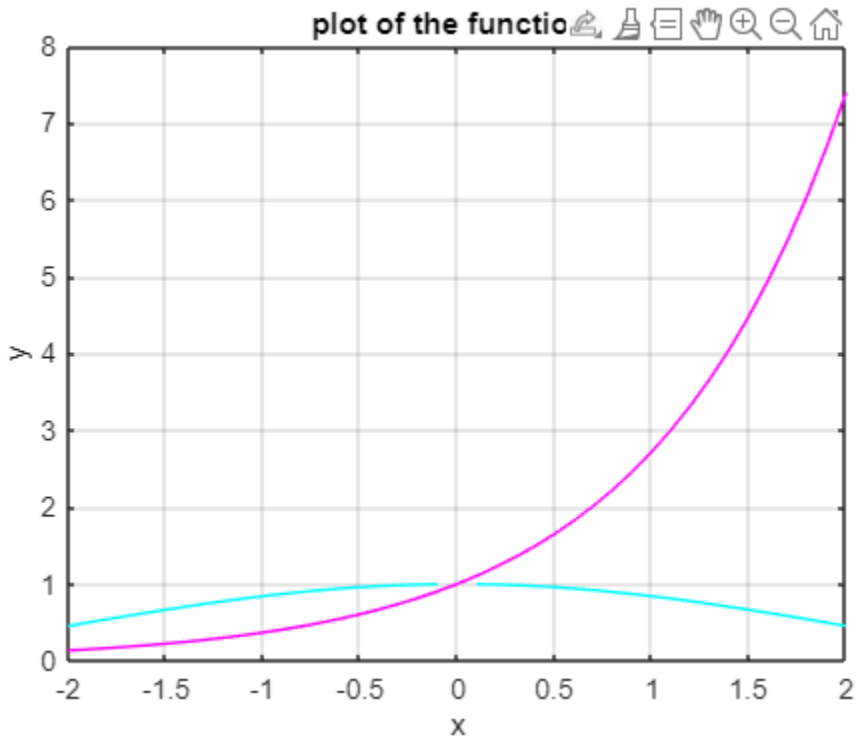
Due to a change in trap2 code for another question, this function is called trap2_new.

```
function I = trap2_new(func,a,b,n)
% trap: composite trapezoidal rule quadrature
%   I = trap(func,a,b,n,pl,p2,...):
%       composite trapezoidal rule
% input:
%   func = name of function to be integrated
%   a, b = integration limits
%   n = number of segments (default = 100)
%   %   I = integral estimate
if nargin<3,error('at least 3 input arguments required'),end
if ~(b>a),error('upper bound must be greater than lower'),end
if nargin<4|isempty(n),n=100;end
x = a; h = (b - a)/n;
s=func(a);
for i = 1 : n-1
  x = x + h;
  s = s + 2*func(x);
end
s = s + func(b);
I = (b - a) * s/(2*n);
end
```

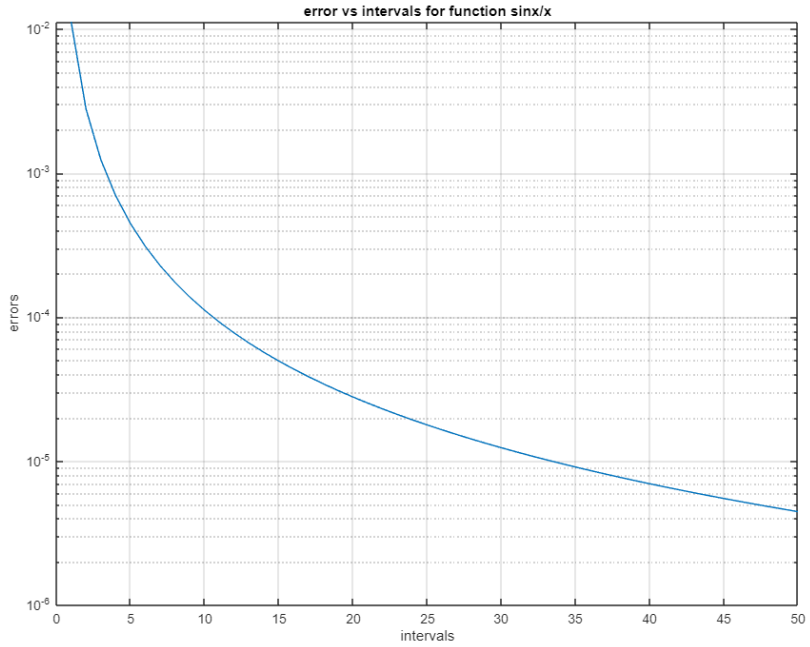
-Main code:

```
clc
clear

x=-2:0.1:2;
y1=my_int1(x);
y2=my_int2(x);
figure(1)
plot(x,y1,'c',x,y2, 'm')
title('plot of the functions')
xlabel('x')
ylabel('y')
grid on
%the reason trp2 is named trap2_new is due to a change that was made to the
%og trap2 code for Q6 so this trap2_new is the original code from the
%lectures
integ1=trap2_new(inline('my_int1(x)'),1,2,10000)
integ2=trap2_new(inline('my_int2(x)'),0,1,10000)

%this is for function1= sinx/x
for k=1:50;
    integ1(k)=trap2_new(inline('my_int1(x)'),1,2,k);
    error1(k)=abs(integ1(k)-0.6593299064);
end
%this is for function2=e^x

for k=1:50;
    integ2(k)=trap2_new(inline('my_int2(x)'),0,1,k);
    error2(k)=abs(exp(1)-1-integ2(k));
end

intervals=1:50;
figure(2)
semilogy(intervals,error1)
title('error vs intervals for function sinx/x')
xlabel('intervals')
ylabel('errors')
grid on
figure(3)
semilogy(intervals, error2)
title('error vs intervals for function e^x')
xlabel('intervals')
ylabel('errors')
grid on
```

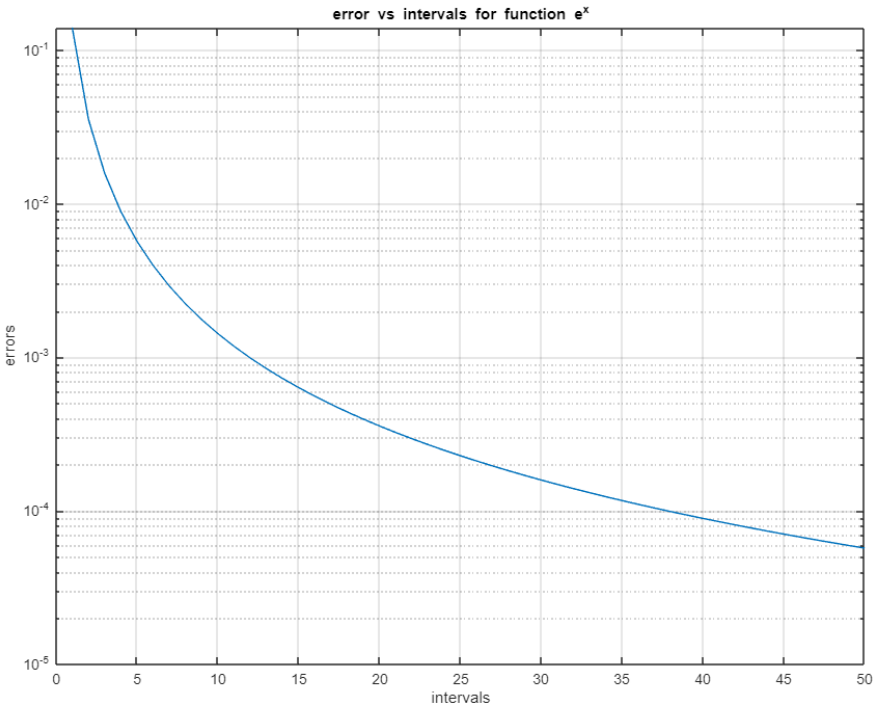
-Plot of the functions:



-Plot of numerical error vs number of segments for sinx/x:



-Plot of numerical error vs number of segments for e^x:



-Output:

```
integ1 =

   0.659329906323676


integ2 =

   1.718281829890868
```

-Comments:

By looking at the graphs, it can be said that by increasing the number of intervals that trap does, the errors get less and less for both functions. Meaning by increasing the number of intervals, we are reaching a better accuracy and making our errors smaller and smaller.

5.2 Romberg Integration

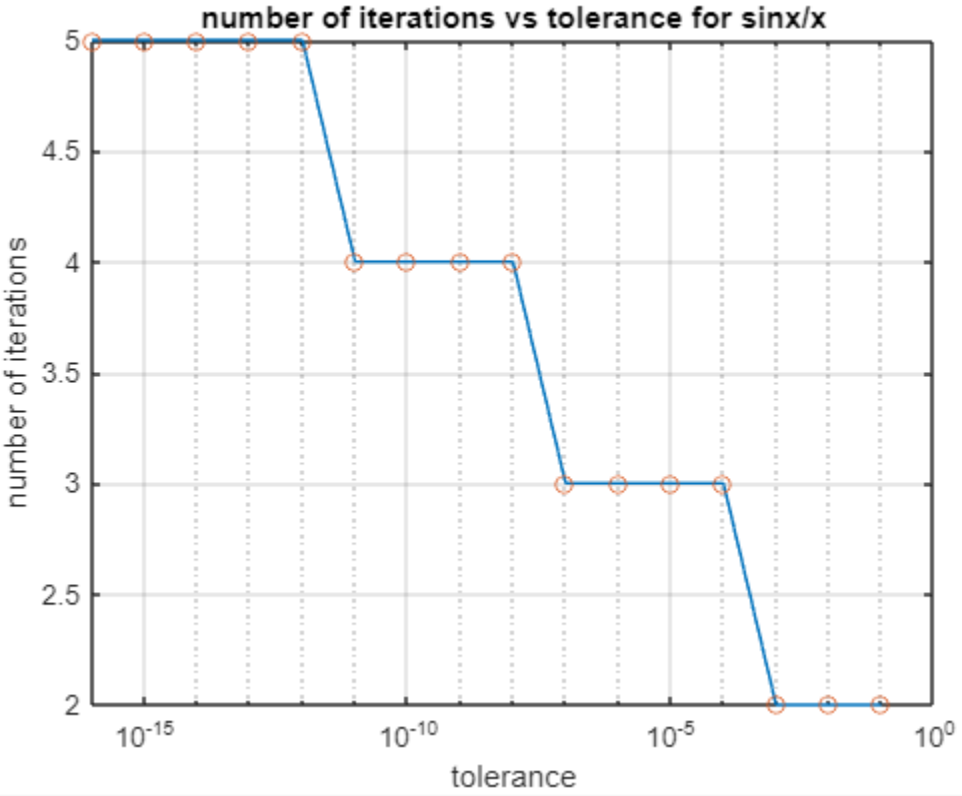
-Romberg function code:

```
function [q,ea,iter]=romberg(func,a,b,es,maxit,varargin)
% romberg: Romberg integration quadrature
%   q = romberg(func,a,b,es,maxit,p1,p2,...):
%         Romberg integration.
% input:
%   func = name of function to be integrated
%   a, b = integration limits
%   es = desired relative error (default = 0.000001%)
%   maxit = maximum allowable iterations (default = 30)
%   p1,p2,... = additional parameters used by func
% output:
%   q = integral estimate
%   ea = approximate relative error (%)
%   iter = number of iterations
if nargin<3,error('at least 3 input arguments required'),end
if nargin<4|isempty(es), es=0.000001;end
if nargin<5|isempty(maxit), maxit=50;end
n = 1;
I(1,1) = trap(func,a,b,n,varargin{:});
iter = 0;
while iter<maxit
  iter = iter+1;
  n = 2^iter;
  I(iter+1,1) = trap(func,a,b,n,varargin{:});
  for k = 2:iter+1
    j = 2+iter-k;
    I(j,k) = (4^(k-1)*I(j+1,k-1)-I(j,k-1))/(4^(k-1)-1);
  end
  ea = abs((I(1,iter+1)-I(2,iter))/I(1,iter+1))*100;
  if ea<=es, break; end
end
q = I(1,iter+1);
end
```

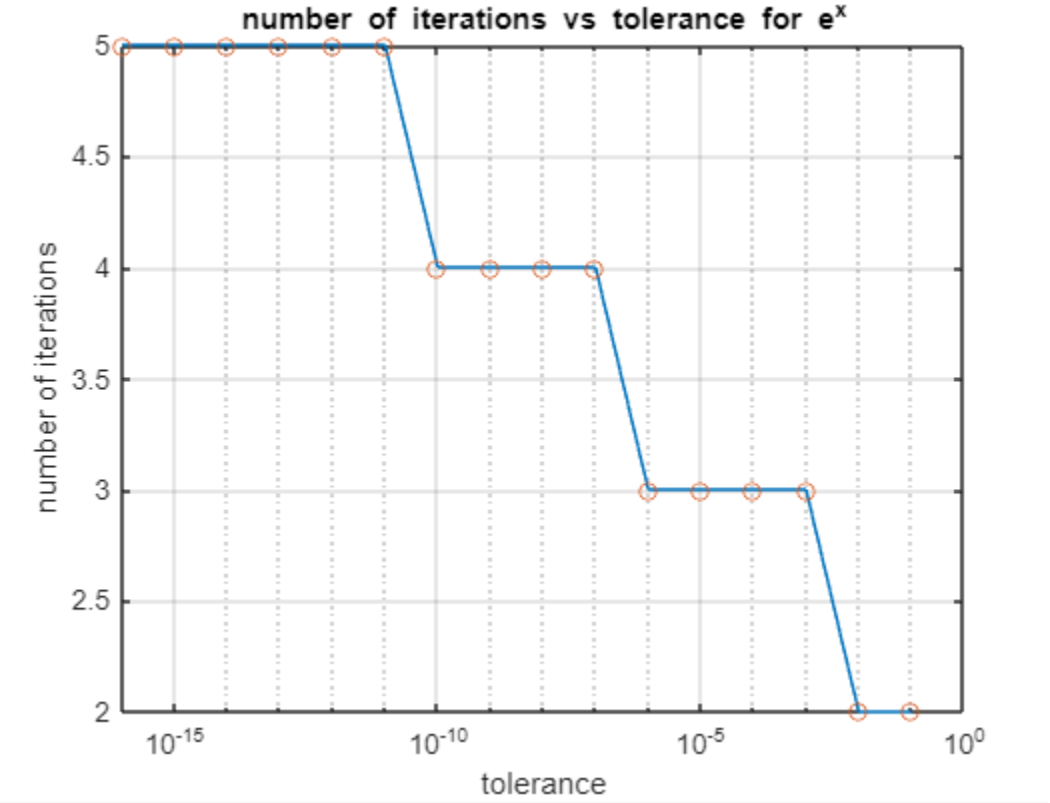
-Main code:

```
clc
clear

x=-2:0.1:2;
y1=my_int1(x);
y2=my_int2(x);
figure(1)
plot(x,y1,'c',x,y2, 'm')
title('plot of the functions')
xlabel('x')
ylabel('y')
grid on
%this is for function 1 sinx/x
tol=10.^[-16:-1];
for k=1:length(tol)
    [q(k), ea(k), iter1(k)]=romberg(inline('my_int1(x)'),1,2,tol(k),40)
end

%this is for function 2 e^x
tol=10.^[-16:-1];
for k=1:length(tol)
    [q(k), ea(k), iter2(k)]=romberg(inline('my_int2(x)'),0,1,tol(k),40)
end

figure(2)
semilogx(tol,iter1,tol,iter1,'o') %plotting for functoin 1
grid on
title('number of iterations vs tolerance for sinx/x')
xlabel('tolerance')
ylabel('number of iterations')
figure(3)
semilogx(tol,iter2,tol,iter2,'o') %plotting for function 2
grid on
title('number of iterations vs tolerance for e^x')
xlabel('tolerance')
ylabel('number of iterations')
```

-Plot number of iterations vs tolerance for the first function:



-Plot number of iterations vs tolerance for the second function:



-Comments:

By comparing the plots from part 2 and 1, we can conclude that Romberg is more accurate than trap method. The reason is that in Romberg to achieve a tolerance of 10^-16, which is a very strict tolerance you do not need a number of iterations more than 5. However, in the first part for trap method, to achieve a tolerance of 10^-4 which is not a very satisfying tolerance, we must have more than 40 number of iterations which comparing to Romberg is a big difference, hence why Romberg is much more reliable and accurate.

Question 5 done.

6. Numerical integration – area between two functions and numerical differentiation

6.1 Trapezoid integration: area between two functions

-trap2 modified function code:

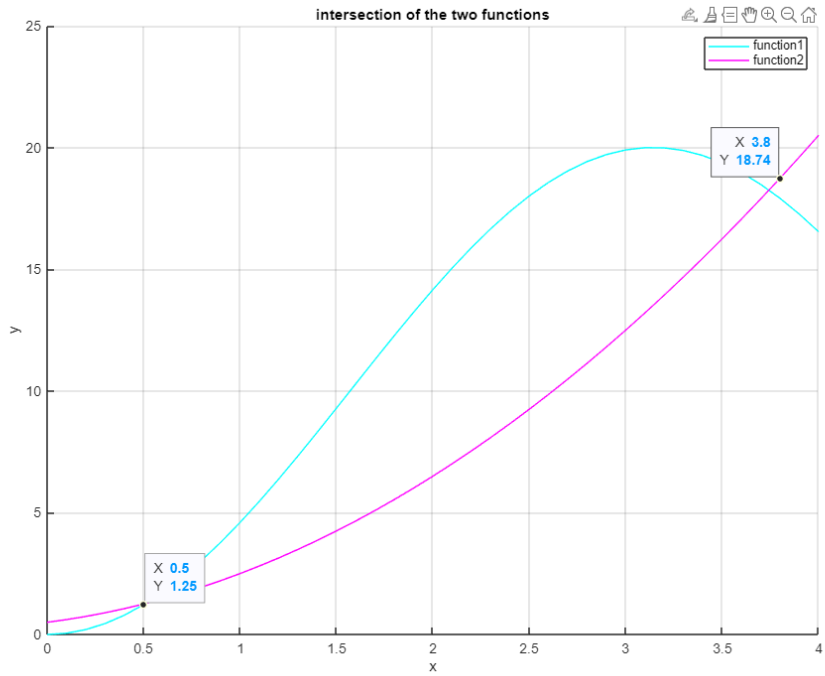
```
function I = trap2(func,a,b,tol)
% trap: composite trapezoidal rule quadrature
%   I = trap(func,a,b,n,pl,p2,...):
%       composite trapezoidal rule
% input:
%   func = name of function to be integrated
%   a, b = integration limits
%   tol = tolerance given
%   %   I = integral estimate
if nargin<3,error('at least 3 input arguments required'),end
if ~(b>a),error('upper bound must be greater than lower'),end
if nargin<4|isempty(tol),tol=1e-6;end %default tolerance is 10^-6
n = 1; %starting with one segment
h = (b - a)/n;
I_prev=h*(func(a)+func(b))/2;
while true
    n = n*2;
    h = (b - a)/n;
    x_mid= a + h : 2 * h : b - h;
    I_curr = I_prev / 2 + h * sum(func(x_mid));

        % Check for convergence
        if abs(I_curr - I_prev) < tol
            break;
        end
        % Update for the next iteration
        I_prev = I_curr;
    end

    I = I_curr; % Final integral estimate
end
```

Main code:

This is part one of the main code for q6.1

```
clc
clear
%these are the codes for the two functions in Q6
x = 0:0.1:4;
hold on
y1=function_1(x); %the functions have already been defined
plot(x,y1,"c")
y2=function_2(x);
plot(x,y2,"m")
legend('function1','function2')
title('intersection of the two functions')
xlabel('x')
ylabel('y')
grid on
hold off

%by looking at graph we can estimate x1 to be around 0.5 and x2 to be 3.7.
%now by using mybisecttol function we try to find x1 and x2.
%to do this we define a new function where f(x)=functio1-function2
%to show how this function works we shall plot the new function
```

-plot of the functions and visual inspections of the intersections:



-Main code:

This is part 2 and 3 of the main code for this question.

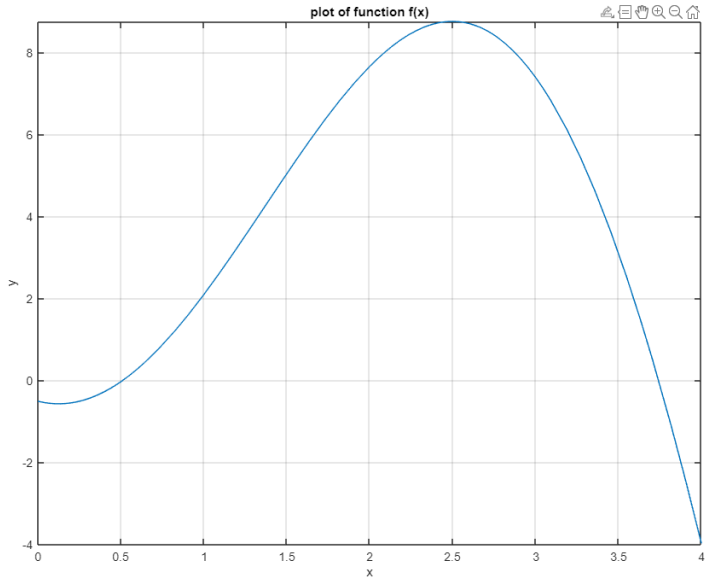
```
clc
clear

x=[0 4];
y=function_1(x);
y=function_2(x);
%by looking at graph we can estimate x1 to be around 0.5 and x2 to be 3.7.
%now by using mybisecttol function we try to find x1 and x2.(?)
%to do this we define a new function where f(x)=functio1-function2
f=@(x) function_1(x)-function_2(x);
%to show how this function works we shall plot the new function
figure(2)
fplot(f,[0 4])
title('plot of function f(x)')
xlabel('x')
ylabel('y')
grid on
%as seen in the graph, where our functions intersect, the new function has
%a root so we use the fzero to find the zeros
x1=fzero('subfunction1',0.5)
x2=fzero('subfunction1',3.5)


clc
clear
x=[0 4];

%now using our modified version of trap we can calculate the are between
%the two functions. knowing the integration rules (will be discussed in the
%lab report), we define a function that is f=function1-function2.
y=subfunction1(x);
%this function has already been defined so we write:
Area_between=trap2(@subfunction1,0.5,3.8,1e-12)

function [y]=subfunction1(x)
y=function_1(x)-function_2(x);
end
```

-Numerical results for x_1 and x_2 and result for the area between two functions:

```
Area_between =

  16.918834295468997
```

```
x1 =

    0.509141299883730


x2 =

    3.742459206947511
```

- *plot of the subfunction:*



- *comments:*

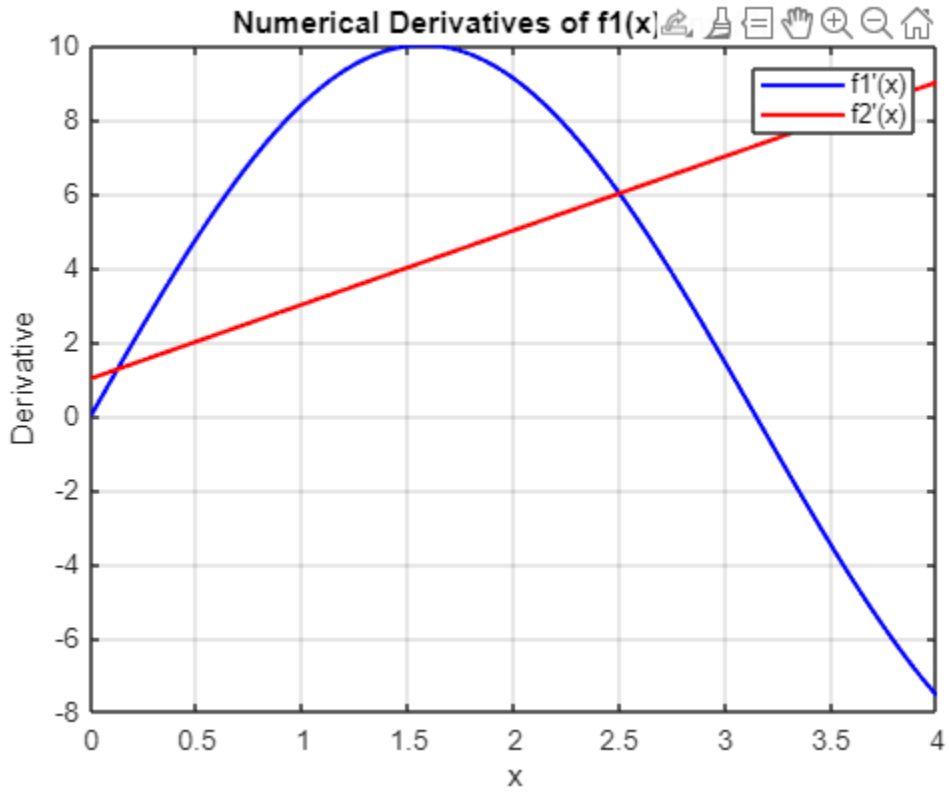
For this task, the tolerance was chosen to be 10^-12, because 10^-15 was a very strict tolerance for this functions and MATLAB wouldn't be able to run the code, so the tolerance was lowered to 10^-12 which is still very strict. Another approach would have been limiting the modified trap2 code so that after a certain number of iterations, it would stop even if it did not reach the given tolerance. However, I decided to lower the tolerance .

6.2 Numerical differentiation

-Code for numerical differentiation:

```
clc
clear

h = 1e-5; % Step size for differentiation
x = linspace(0, 4, 100); %defining x values

% Numerical derivatives
df1_dx = (function_1(x + h) - function_1(x - h)) / (2 * h);
df2_dx = (function_2(x + h) - function_2(x - h)) / (2 * h);

figure
plot(x, df1_dx, 'b', 'LineWidth', 1.5); % Plot derivative of f1
hold on
plot(x, df2_dx, 'r', 'LineWidth', 1.5); % Plot derivative of f2
hold off

xlabel('x')
ylabel('Derivative')
legend('f1''(x)', 'f2''(x)')
title('Numerical Derivatives of f1(x) and f2(x)')
grid on
```

-Plot of the first derivatives of the functions:



-Comments on possible difficulties:

The first issue with this method is accuracy. Since we are subtracting such close values, it can lead to errors such as rounding issues or if we choose a step size that is too large, we might face a lot of errors. Also, if h is too small, we might have less truncation errors, but we will have more rounding errors. Overall, this method is not aa bad method for easy and not complex functions, but as we go towards more complex functions this code will have more accuracy issues.

Question 6 done.